

Sprint 15 Structured Notes: Final Integration Project

1. Current Progress

You have now reached the **final sprint** of the Relational Databases learning path.

The full path has **15 sprints**. The curriculum says each sprint can take about **1 to 3 days**, and you should **not skip checkpoints**.

What you have already learned

Sprint 1 to Sprint 4

You learned the beginner foundation tools:

- VS Code
- terminal basics
- Bash commands
- Bash scripting

Sprint 5 to Sprint 10

You learned Git and GitHub workflow:

- Git setup
- local Git workflow
- GitHub repositories
- branches
- pull requests
- `.gitignore`
- merge conflicts
- safer Git tools
- beginner CI/CD ideas

Sprint 11 to Sprint 13

You learned database and SQL foundations:

- PostgreSQL
- SQL basics
- tables
- keys
- relationships

- joins

Sprint 14

You learned database quality topics:

- normalization
- SQL injection prevention
- N+1 problem

Simple meaning

Up to Sprint 14, you learned the important pieces one by one.

Sprint 15 is where you combine everything into one small real project.

2. What Sprint 15 Is

Sprint 15 is the **Final Integration Project**.

This sprint is not mainly about learning a brand-new theory topic. It is about **using the full toolchain together in one beginner project**.

Main idea

You will build one small database project from beginning to end.

That means you will use:

- tools
- terminal commands
- Git
- GitHub
- PostgreSQL
- SQL
- normalization ideas
- documentation

Simple meaning

Sprint 15 is the project sprint that proves you can use what you learned in the earlier sprints.

3. Main Goal of Sprint 15

The main goal is:

Build one small database project from start to finish.

Project name

beginner-inventory-db

Tools used in the project

- VS Code
- Bash
- Git
- GitHub
- PostgreSQL
- SQL
- Normalization
- README documentation

Simple meaning

In Sprint 15, you are building a complete beginner project instead of only learning one isolated topic.

4. What You Are Building

You are building a small **inventory database**.

What inventory means

Inventory means:

- stock
- items
- products available in a business

Example idea

A shop may need to track:

- products
- categories
- suppliers
- stock movements

Questions this database should answer

Your project should help answer questions like:

- What products do we have?
- Which category does each product belong to?
- Who supplies each product?
- How much stock came in?
- How much stock went out?

Simple meaning

This project is about storing and reading product stock data in an organized way.

5. Main Tables in the Project

The project uses four main tables:

- categories
- suppliers
- products
- stock_movements

Each table has a different job.

6. Table 1: categories

This table stores product groups.

Example

category_id	category_name
1	Electronics
2	Stationery
3	Food

What it means

- category_id = the unique ID of the category
- category_name = the category name

Simple meaning

A category is a group that products belong to.

Examples:

- Laptop → Electronics
 - Pen → Stationery
-

7. Table 2: suppliers

This table stores the suppliers of products.

Example

supplier_id	supplier_name
1	ABC Suppliers
2	City Traders

What it means

- `supplier_id` = the unique ID of the supplier
- `supplier_name` = the name of the supplier

Simple meaning

A supplier is the person or company that gives products to the shop or business.

8. Table 3: products

This table stores product details.

Example

product_id	product_name	category_id	supplier_id	price
1	Laptop	1	1	250000
2	Pen	2	2	100

What it means

- `product_id` = unique ID of the product

- `product_name` = product name
- `category_id` = link to the category table
- `supplier_id` = link to the supplier table
- `price` = product price

Foreign keys in this table

Inside `products`:

- `category_id` is a **foreign key**
- `supplier_id` is a **foreign key**

These foreign keys connect the `products` table to other tables.

Simple meaning

Each product belongs to:

- one category
- one supplier

9. Table 4: `stock_movements`

This table stores stock changes.

Example

<code>movement_id</code>	<code>product_id</code>	<code>movement_type</code>	<code>quantity</code>	<code>movement_date</code>
1	1	IN	10	2026-06-02
2	1	OUT	2	2026-06-03

What it means

- `movement_id` = unique ID of the stock movement
- `product_id` = link to the product table
- `movement_type` = whether stock came in or went out
- `quantity` = how much stock moved
- `movement_date` = date of the movement

Meaning of movement types

- `IN` = stock added
- `OUT` = stock removed

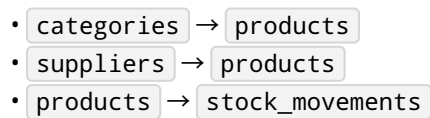
Simple meaning

This table records stock history. It helps the system remember when stock was added or removed.

10. How the Tables Work Together

These tables are connected.

Relationship flow



What this means

- One category can have many products
- One supplier can have many products
- One product can have many stock movements

Relationship type

This is mostly **one-to-many** relationship design.

Simple meaning

The tables are not isolated. They are linked so the database can understand the full story of each product.

11. Why This Design Is Better

A bad design would try to keep everything in one large table.

Bad example

product_name	category_name	supplier_name	stock_quantity
--------------	---------------	---------------	----------------

Why this is bad

This is bad because:

- category names repeat many times
- supplier names repeat many times
- the table becomes messy

- updates become harder
- the design is less organized

Better design

Separate the data into:

- categories
- suppliers
- products
- stock_movements

Simple meaning

Each table should do one job instead of mixing many jobs together.

12. Normalization in Sprint 15

Sprint 15 uses the normalization idea you learned earlier.

Goal

The project design should help you reach at least **3NF**.

Why that matters

A cleaner design:

- reduces repetition
- makes updates easier
- makes the database more organized
- makes relationships clearer

Simple meaning

The project is not only about making tables. It is also about making the tables **clean and well-structured**.

13. Step-by-Step Sprint 15 Workflow

Sprint 15 follows a real project workflow.

You will: 1. create a project folder 2. create files and folders 3. initialize Git 4. add `.gitignore` 5. create a PostgreSQL database 6. create tables 7. insert sample data 8. write queries 9. document the project 10. use GitHub branch and pull request workflow

Simple meaning

Sprint 15 is a complete project process, not just a single SQL task.

14. Step 1: Create the Project Folder

Use Bash commands:

```
mkdir beginner-inventory-db  
cd beginner-inventory-db
```

What these commands mean

- `mkdir beginner-inventory-db` = create the project folder
- `cd beginner-inventory-db` = move into that folder

Simple meaning

First, you create the project folder and go inside it.

15. Step 2: Create the Project Files

Create folders and files using Bash:

```
mkdir sql docs  
touch README.md  
touch .gitignore  
touch sql/schema.sql  
touch sql/insert_data.sql  
touch sql/queries.sql  
touch docs/design-explanation.md
```

What this creates

Main files

- `README.md`
- `.gitignore`

SQL folder

- `sql/schema.sql`

- `sql/insert_data.sql`
- `sql/queries.sql`

Docs folder

- `docs/design-explanation.md`

Project structure

```
beginner-inventory-db/  
|  
├─ README.md  
├─ .gitignore  
|  
├─ sql/  
|   ├─ schema.sql  
|   ├─ insert_data.sql  
|   └─ queries.sql  
└─ docs/  
    └─ design-explanation.md
```

What each file is for

`README.md`

Explains the project.

`.gitignore`

Tells Git which files should not be tracked.

`schema.sql`

Stores the SQL used to create tables.

`insert_data.sql`

Stores the sample data insert commands.

`queries.sql`

Stores the SQL queries for reading data.

```
design-explanation.md
```

Stores the written explanation of the database design.

Simple meaning

This step organizes the project clearly before you start writing SQL.

16. Step 3: Initialize Git

Use these commands:

```
git init
git status
git add .
git commit -m "chore: create project structure"
```

What these commands mean

```
git init
```

Start Git tracking in the project folder.

```
git status
```

Show the current Git state.

```
git add .
```

Stage the current files.

```
git commit -m "chore: create project structure"
```

Create the first commit and save the project structure in Git history.

Why this matters

This gives you your first clean Git checkpoint.

Simple meaning

Git begins tracking the project from the beginning.

17. Step 4: Add `.gitignore`

Inside `.gitignore`, add:

```
.env  
*.log  
.DS_STORE  
Thumbs.db
```

What these mean

- `.env` = environment or secret file
- `*.log` = log files
- `.DS_STORE` = macOS system file
- `Thumbs.db` = Windows system file

Why this matters

These files are usually:

- private
- unnecessary
- system-generated
- not useful for the project repository

Simple meaning

`.gitignore` tells Git not to track unwanted files.

18. Step 5: Create the PostgreSQL Database

Open `psql`, then run:

```
CREATE DATABASE beginner_inventory_db;
```

Then connect to it:

```
\c beginner_inventory_db
```

What this means

```
CREATE DATABASE beginner_inventory_db;
```

Create a new database called `beginner_inventory_db`.

```
\c beginner_inventory_db
```

Connect to that database.

Simple meaning

You create the database container and enter it so you can start creating tables.

19. Step 6: Create the Tables

Inside `sql/schema.sql`, write the SQL for the tables.

Example schema

```
CREATE TABLE categories (  
    category_id SERIAL PRIMARY KEY,  
    category_name VARCHAR(100) NOT NULL  
);  
  
CREATE TABLE suppliers (  
    supplier_id SERIAL PRIMARY KEY,  
    supplier_name VARCHAR(100) NOT NULL  
);  
  
CREATE TABLE products (  
    product_id SERIAL PRIMARY KEY,  
    product_name VARCHAR(100) NOT NULL,  
    category_id INTEGER NOT NULL,  
    supplier_id INTEGER NOT NULL,  
    price DECIMAL(10, 2),  
    FOREIGN KEY (category_id) REFERENCES categories(category_id),  
    FOREIGN KEY (supplier_id) REFERENCES suppliers(supplier_id)  
);  
  
CREATE TABLE stock_movements (  
    movement_id SERIAL PRIMARY KEY,  
    product_id INTEGER NOT NULL,  
    movement_type VARCHAR(10) NOT NULL,  
    quantity INTEGER NOT NULL,
```

```
movement_date DATE NOT NULL,  
FOREIGN KEY (product_id) REFERENCES products(product_id)  
);
```

Important concepts inside this schema

SERIAL PRIMARY KEY

Used for automatic numbering and unique row IDs.

VARCHAR(100)

Used for text values with a maximum length.

INTEGER

Used for whole numbers.

DECIMAL(10, 2)

Used for price values with decimal places.

DATE

Used for dates.

NOT NULL

Means the column must have a value.

FOREIGN KEY ... REFERENCES

Connects one table to another table.

Simple meaning

This step builds the structure of the database.

20. Step 7: Insert Sample Data

Inside `sql/insert_data.sql`, add sample rows.

Example inserts

```
INSERT INTO categories (category_name)
VALUES
('Electronics'),
('Stationery');

INSERT INTO suppliers (supplier_name)
VALUES
('ABC Suppliers'),
('City Traders');

INSERT INTO products (product_name, category_id, supplier_id, price)
VALUES
('Laptop', 1, 1, 250000.00),
('Pen', 2, 2, 100.00);

INSERT INTO stock_movements (product_id, movement_type, quantity, movement_date)
VALUES
(1, 'IN', 10, '2026-06-02'),
(1, 'OUT', 2, '2026-06-03'),
(2, 'IN', 100, '2026-06-02');
```

What this data shows

- categories are added first
- suppliers are added next
- products are added after that
- stock movement records are added last

Why this order makes sense

Because products depend on categories and suppliers. And stock movements depend on products.

Simple meaning

This step puts real practice data into the database.

21. Step 8: Write SQL Queries

Inside `sql/queries.sql`, write useful SQL queries.

This helps prove that your tables and relationships work correctly.

22. Query 1: Show All Products

```
SELECT *  
FROM products;
```

What it does

This reads all rows and columns from the `products` table.

Simple meaning

Show all product records.

23. Query 2: Show Products with Category and Supplier

```
SELECT  
    products.product_name,  
    categories.category_name,  
    suppliers.supplier_name,  
    products.price  
FROM products  
JOIN categories  
    ON products.category_id = categories.category_id  
JOIN suppliers  
    ON products.supplier_id = suppliers.supplier_id;
```

What it does

This query combines data from:

- `products`
- `categories`
- `suppliers`

Why it matters

This proves the foreign key relationships are working.

What result it should show

For each product, it shows:

- product name

- category name
- supplier name
- price

Simple meaning

This query joins connected tables so you can read useful combined information.

24. Query 3: Show Stock Movement History

```
SELECT
    products.product_name,
    stock_movements.movement_type,
    stock_movements.quantity,
    stock_movements.movement_date
FROM stock_movements
JOIN products
    ON stock_movements.product_id = products.product_id;
```

What it does

This query combines:

- stock_movements
- products

What result it should show

It shows:

- product name
- whether stock came in or went out
- quantity
- movement date

Simple meaning

This query shows the stock history of each product.

25. Why SELECT and JOIN Matter in Sprint 15

Sprint 15 does not only ask you to create tables. It also expects you to prove that the design works by reading data back correctly.

Important ideas here

- `SELECT` reads data
- `JOIN` combines related data from multiple tables
- foreign keys make joins possible

Simple meaning

A database project is not complete if you only create tables. You must also show that you can query them properly.

26. Step 9: Create a Documentation Branch

Now use GitHub workflow with a branch.

Create a branch:

```
git checkout -b docs/update-readme
```

Then add and commit documentation:

```
git add README.md docs/design-explanation.md
git commit -m "docs: add project documentation"
git push -u origin docs/update-readme
```

Then open a pull request on GitHub.

What this means

```
git checkout -b docs/update-readme
```

Create and switch to a new documentation branch.

```
git add README.md docs/design-explanation.md
```

Stage the documentation files.

```
git commit -m "docs: add project documentation"
```

Save the documentation work as a commit.

```
git push -u origin docs/update-readme
```

Push the new branch to GitHub.

Simple meaning

Sprint 15 also checks that you can use a real GitHub workflow, not only SQL.

27. Pull Request Workflow in Sprint 15

After pushing the documentation branch, go to GitHub and open a pull request.

Workflow summary

```
branch → commit → push → pull request → merge
```

What this means

- branch = separate work area
- commit = save your work
- push = upload your work to GitHub
- pull request = request to merge your changes
- merge = add the work into the main branch

Simple meaning

This step proves you can use the Git workflow you learned earlier.

28. Step 10: Write the README

Your `README.md` should explain the project clearly.

It should include

- project name
- project purpose
- tables used
- relationships
- how to run SQL files
- example queries
- what you learned

Example structure

```
# Beginner Inventory DB

This is a beginner PostgreSQL inventory database project.

## Tables
- categories
- suppliers
- products
- stock_movements

## Relationships
- One category has many products.
- One supplier has many products.
- One product has many stock movements.

## Files
- sql/schema.sql creates the tables.
- sql/insert_data.sql inserts sample data.
- sql/queries.sql contains SELECT and JOIN queries.
```

Why README matters

README is the front-page explanation of the project. It helps another person understand:

- what the project is
- what files exist
- how the project works

Simple meaning

README explains the project in a clean human-readable way.

29. Step 11: Write the Design Explanation

The project also includes:

- `docs/design-explanation.md`

What this file should explain

- why you chose these tables
- how the relationships work
- why the design is normalized

- why this design is better than one big messy table

Simple meaning

This file explains your database thinking, not just your SQL code.

30. Sprint 15 Checkpoint

You pass Sprint 15 when you can submit these things:

1. GitHub repository link
2. SQL file
3. README file
4. Short explanation of your database design

What this shows

If you can submit those items, it means you can:

- organize a project folder
- write SQL files
- build a database
- add sample data
- query the data
- document the project
- use Git and GitHub workflow

Simple meaning

The checkpoint proves that you can complete a small real database project from start to finish.

31. Full Skill Integration in Sprint 15

Sprint 15 brings many earlier sprints together.

Tool integration

From early sprints

- folder creation
- terminal use
- Bash commands
- project structure

From Git sprints

- Git tracking
- commits
- branches
- push
- pull request
- GitHub repository workflow

From database sprints

- PostgreSQL database creation
- SQL table creation
- foreign keys
- insert statements
- SELECT queries
- JOIN queries
- normalization thinking

From documentation skills

- README writing
- design explanation writing

Simple meaning

Sprint 15 is the place where everything connects together.

32. Key Concepts Covered in Sprint 15

Here is a clean concept list from Sprint 15.

Project concepts

- final integration project
- complete beginner database project
- inventory database

Tool concepts

- VS Code
- Bash
- Git
- GitHub
- PostgreSQL
- SQL
- README

Database design concepts

- categories table
- suppliers table
- products table
- stock_movements table
- foreign key relationships
- one-to-many relationships
- normalized design
- 3NF target idea

SQL concepts used in the project

- CREATE DATABASE
- \c
- CREATE TABLE
- SERIAL PRIMARY KEY
- VARCHAR
- INTEGER
- DECIMAL
- DATE
- NOT NULL
- FOREIGN KEY
- REFERENCES
- INSERT INTO
- VALUES
- SELECT
- JOIN

Git concepts used in the project

- git init
- git status
- git add
- git commit
- git checkout -b
- git push -u origin
- pull request
- merge

Documentation concepts

- README.md
 - .gitignore
 - design-explanation.md
-

33. Final Beginner Summary

Sprint 15 is the final project sprint.

You are not only writing a few SQL commands. You are doing the full workflow:

- create the project folder
- create the files
- initialize Git
- create the database
- build normalized tables
- insert sample data
- write SELECT and JOIN queries
- document the project
- use branch and pull request workflow
- prepare final submission

Final simple meaning

Sprint 15 proves that you can build a small real database project from beginning to end using the tools and concepts from the whole course.